Reconstruction de l'environnement de build

Équipe projet openwebui-android

Comment reconstruire **intégralement** l'environnement de build du projet Open WebUI Android à partir du seul dépôt git, sur une machine neuve : outils, chaînes de compilation, sources verrouillées et wheels natifs. Un script unique (`scripts/setup-env.sh`) reproduit tout dans le répertoire du projet, sans installation globale.

7/11/2026

Sommaire

1. Principe 3
2. Prérequis de la machine 4
3. Ce que fait `setup-env.sh` 5
4. Versionné vs régénéré 6
5. Reprendre le projet après reconstruction 7
6. Dépannage du bootstrap 8
7. Traçabilité 9

1. Principe

Le dépôt git contient **le projet** : sources, patchs, scripts, documentation, wrappers de compilation et wheels natifs pré-construits. Tout ce qui est **téléchargeable** (SDK, JDK, NDK, Rust, qemu, Python, sources amont) est reconstruit par un script unique :

```
git clone <dépôt> openwebui-android
cd openwebui-android
bash scripts/setup-env.sh --build
```

À la fin : `dist/app-debug.apk` installable, et `bash scripts/setup-env.sh --check` pour auditer l'existant à tout moment.

> **Attention**
>
> Règle d'isolation : le script n'écrit **jamais** en dehors du projet (ni `~/.profile`, ni `~/.cargo`, ni paquets système). Toute intervention manuelle doit commencer par `source scripts/env.sh`.

2. Prérequis de la machine

Élément	Détail
Système	Linux **aarch64** (le projet a été construit sous Debian 13 / proot) ; x86_64 possible mais exige d'adapter les URL de téléchargement
Outils	`git`, `curl`, `unzip`, `dpkg-deb`, `python3` **3.13.x**
Espace	≈ 20 Go libres pendant le build (environnement fini ≈ 13 Go dont SDK/NDK)
Réseau	≈ 3 Go de téléchargements ; durée indicative 20-40 min
RAM	8 Go conseillés (Gradle limité à 2,5 Go dans `gradle.properties`)

3. Ce que fait `setup-env.sh`

N°	Étape	Destination dans le projet
1	Python 3.12 portable (copie locale si présente, sinon python-build-standalone)	`tools/python312/`
2	venv Python 3.13 + `requirements-min.txt` + wheel/setuptools/maturin	`.venv313/`
3	JDK 21 Temurin	`tools/jdk/`
4	Android SDK : cmdline-tools, platform-34, build-tools 34	`android-sdk/`
5	Gradle 8.10.2	`tools/gradle/`
6	NDK r27c (sysroot pour le cross-build Rust)	`tools/android-ndk/`
7	Rust + cible `aarch64-linux-android` (`--no-modify-path`)	`tools/cargo/` , `tools/rustup/`
8	Python Android (PyO3, artefact Maven), qemu-user-static + sysroot amd64	`tools/pyo3/` , `tools/qemu/`
9	Sources verrouillées : wheel Open WebUI pristine + SearXNG au commit figé	`vendor/unpacked/` , `vendor/searxng/`
10	Cross-build des 7 wheels natifs (`build-wheels.sh`)	`wheels/`
11	Régénération de `src/` (patchs P1-P11)	`src/`
12	Optionnel `--build` : APK complet	`dist/app-debug.apk`

> Les **wrappers de compilation** (`tools/ndk-cc/` , `tools/wrappers/`) et `libunwind.a` sont versionnés dans le dépôt : ce sont des éléments **écrits**, pas téléchargeables. Idem pour `scripts/` , `app/` , `docs/` et les wheels de `wheels/` (pré-construits pour gagner une demi-heure de compilation).

4. Versionné vs régénéré

Contenu	Dans git ?	Récupération
`app/`, `scripts/`, `docs/`, `*.md`, configurations Gradle	oui	—
`tools/ndk-cc/`, `tools/wrappers/`	oui	—
`wheels/` (7 wheels Android)	oui	reconstruit par `build-wheels.sh` si besoin
`src/` (code embarqué patché)	non	`prepare_src.py` à partir de `vendor/`
`vendor/` (Open WebUI, SearXNG)	non	étape 9 du script (pins documentés)
`android-sdk/`, `tools/` (JDK, NDK, Gradle, Rust, qemu, python)	non	étapes 1-8 du script
caches (`.gradle-cache`, `.pip-cache`, `.tmp`…)	non	recréés au besoin
`dist/` (APK)	non	`build.sh` — l'APK n'est **jamais** versionné (limite GitHub : 100 Mo par fichier ; binaire rebuildable) ; livré hors dépôt

5. Reprendre le projet après reconstruction

1. `bash scripts/setup-env.sh --check` — vérifier ce qui manque ;
2. `bash scripts/smoke.sh` — test de fumée : le serveur doit répondre sur `http://127.0.0.1:8080/health` et SearXNG sur le port 8888 ;
3. `bash scripts/build.sh` — builder l'APK (`dist/app-debug.apk`) ;
4. lire `docs/maintenance.pdf` pour la maintenance courante et les mises à jour, et `journal.md` pour l'état des décisions.

> **Bonne pratique**
>
> Pour un projet déjà présent mais dont l'environnement a été perdu : `bash scripts/setup-env.sh` suffit — il est idempotent et ne rebat que ce qui manque.

6. Dépannage du bootstrap

Symptôme	Cause et correctif
`'git'/'curl'/'unzip' manquant`	prérequis machine non satisfait — installer via la gestion de paquets **du système** (hors périmètre du projet) ou changer de machine
Le venv 3.13 refuse de se créer	la commande `python3` n'est pas en 3.13 : mettre un python3.13 en tête de `PATH`
Échec `sdkmanager` (licences)	relancer l'étape 4 ; `JAVA_HOME` doit pointer sur `tools/jdk`
L'étape 9 échoue sur Open WebUI	PyPI inaccessible ou version retirée : déposer manuellement le wheel dans `vendor/wheel/` puis relancer
L'étape 9 échoue sur SearXNG	le commit figé n'est plus accessible : `git clone` puis choisir un commit **vérifié** (sans `curl_cffi`, cf. `maintenance.pdf` §8) et mettre à jour `VENDOR_COMMIT` dans le script
Cross-build Rust en échec (`-lpython3.13`, `libunwind`)	vérifier `tools/pyo3/prefix/lib` et `tools/ndk-cc/lib/libunwind.a` ; wrappers bien exécutables (`chmod +x`)
qemu : `required file not found`	le sysroot amd64 est incomplet : les liens `lib` et `lib64` doivent pointer vers `usr/lib*`
Espace insuffisant en fin de build	libérer `downloads/` (≈ 3 Go d'archives) : les étapes n'en ont plus besoin

7. Traçabilité

Toute reconstruction ou modification de l'environnement doit être consignée dans `journal.md` (date, étape touchée, raison). Les versions verrouillées sont rappelées dans `docs/maintenance.pdf` (annexe « Versions verrouillées ») et dans `requirements-min.txt`.